

Estructuras de Datos

Clase 15 – Árboles binarios



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

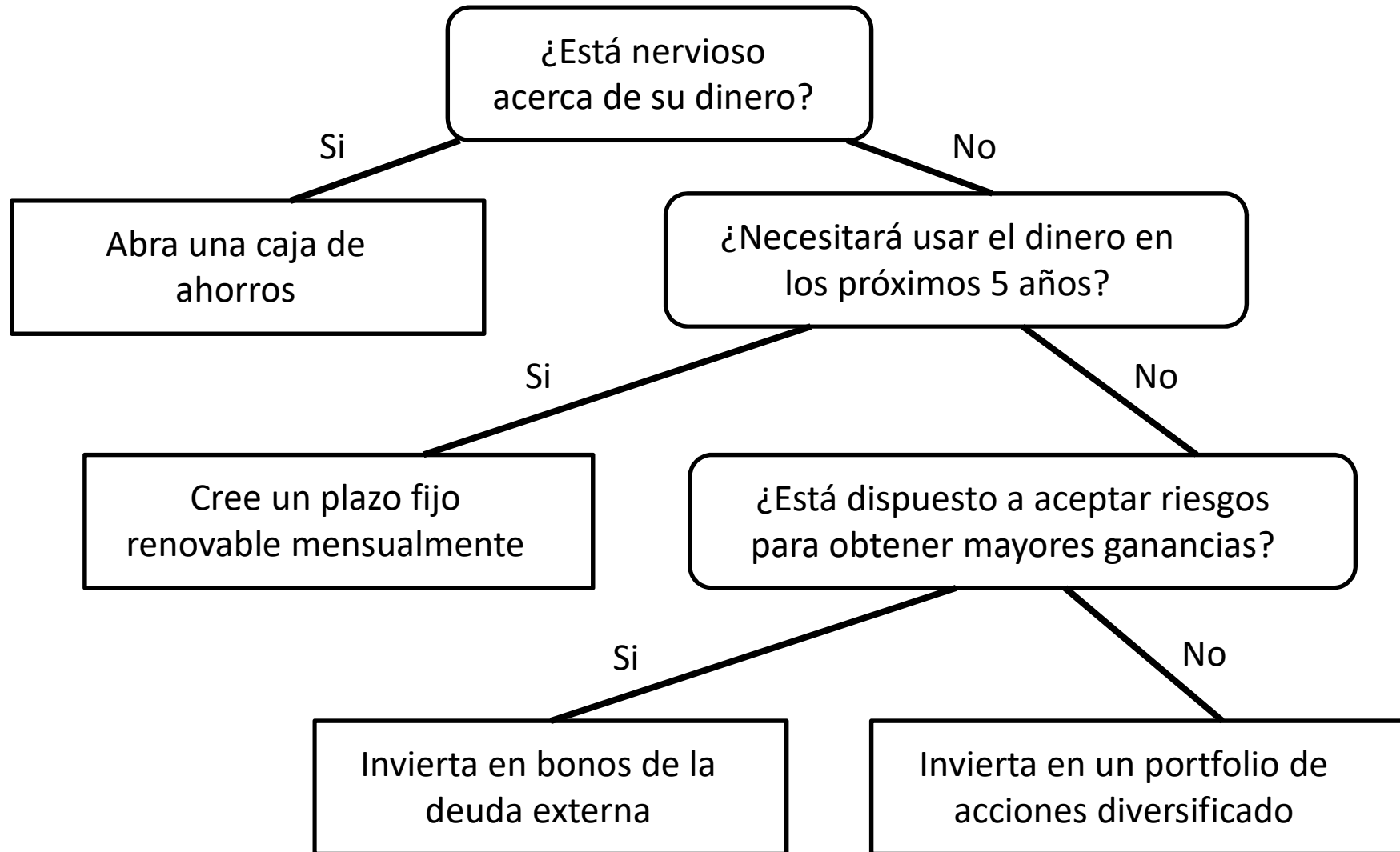
Árboles binarios

- Un árbol binario es un árbol ordenado que cumple:
 - 1) Cada nodo tiene a lo sumo dos hijos
 - 2) Cada nodo hijo es o bien hijo izquierdo o hijo derecho
 - 3) El hijo izquierdo precede al hijo derecho en el orden de los hijos de un nodo
- El subárbol que tiene como raíz al hijo izquierdo se llama subárbol izquierdo.
- El subárbol que tiene como raíz al hijo derecho se llama subárbol derecho.
- En un árbol binario propio, cada nodo tiene 0 o dos hijos (GT también le dice full BT).
- Si un árbol binario no es propio, entonces es impropio.

Aplicaciones: Árboles de decisión

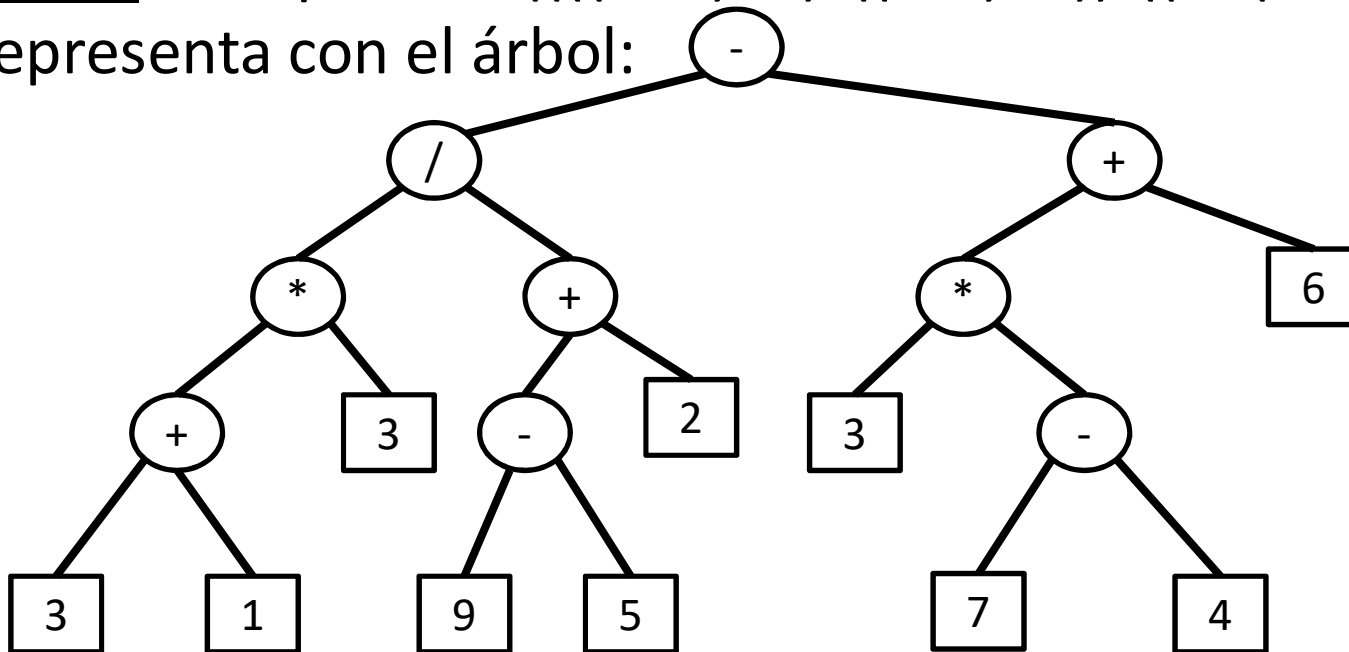
- Se desea representar los resultados asociados a un conjunto de preguntas con respuesta “Si” o “No”
- Cada nodo interno se asocia con una pregunta.
- Se comienza en la raíz y con cada pregunta se va a la izquierda o a la derecha dependiendo de si la respuesta a la pregunta es “si” o “no”.
- Cuando se llega a una hoja, se tiene un resultado al cual se llega partir de las respuestas dadas en los ancestros de la hoja.

Aplicaciones: Árboles de decisión



Aplicaciones: Expresiones aritméticas

- Una expresión aritmética puede representarse con un árbol binario.
- Las hojas almacenan constantes o variables
- Los nodos internos almacenan los operadores +, -, * y /.
- Ejemplo: La expresión $((((3+1)*3)/((9-5)+2))-((3*(7-4))+6))$ se representa con el árbol:



Definición recursiva de árbol binario

Un árbol binario T es o bien vacío, o consiste de:

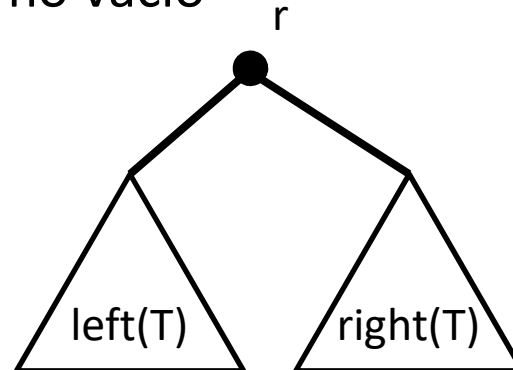
- Un nodo r , llamado la raíz de T , que contiene un rótulo (o elemento)
- Un árbol binario, llamado el subárbol izquierdo de T
- Un árbol binario, llamado el subárbol derecho de T

Caso base:

$T = \text{Árbol vacío}$

Caso recursivo:

$T = \text{Árbol no vacío}$



Definición recursiva de árbol binario (no vacío)

Un árbol binario T es :

Hoja(k): Un nodo r , llamado la raíz de T , que contiene un rótulo (o elemento k)

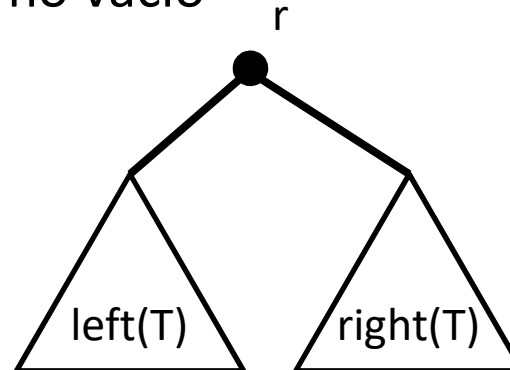
Nodo(k, T_l, T_d): k es el rótulo y T_l y T_d son árboles binarios llamados hijo izquierdo y derecho, resp.

Caso base:



Caso recursivo:

$T = \text{Árbol no vacío}$



Obtención del árbol binario de expresión aritmética

Ejemplo: $\text{exp} = 5 + 6 - 7$ (expresión solo formada por + y -)

Algoritmo Parse(exp)

Si exp no contiene un operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

$\text{op} \leftarrow \text{último_operador}(\text{exp})$

$T_1 \leftarrow \text{Parse}(\text{izquierda}(\text{exp}, \text{op}))$

$T_2 \leftarrow \text{Parse}(\text{derecha}(\text{exp}, \text{op}))$

retornar un árbol binario con op como rótulo de la raíz
y T_1 y T_2 como hijos izquierdo y derecho resp.

Nota:

$5 + 6 - 7 = (5 + 6) - 7 = 11 - 7 = 4 \text{ ok}$

$5 + (6 - 7) = 5 + (-1) = 4$

$7 - 2 - 4 = (7 - 2) - 4 = 5 - 4 = 1 \text{ ok}$

$7 - (2 - 4) = 7 - (-2) = 7 + 2 = 9$

que no es lo que quiero.

Ultimo_operador(exp) = busca el último operador de la expresión exp recorriendo de derecha a izquierda. En el caso de $\text{exp}=5+6-7$ retorna -.

Izquierda(exp,op) = retorna la porción izquierda de exp considerando la ubicación de op.

En el caso de $\text{exp}=5+6-7$ y $\text{op}=-$ retorna 5+6.

Derecha(exp,op) = retorna la porción derecha de exp considerando la ubicación de op.

En el caso de $\text{exp}=5+6-7$ y $\text{op}=-$ retorna 7

Obtención del árbol binario de expresión aritmética sin paréntesis

Ejemplo: $\text{exp} = 5 + 6 - 7$ (expresión solo formada por + y -)

Algoritmo Parse(exp)

Si exp no contiene un operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

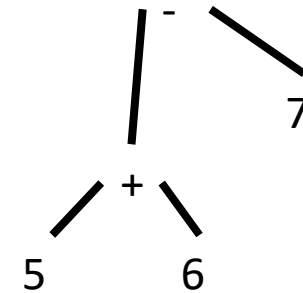
Sino

$\text{op} \leftarrow \text{último_operador}(\text{exp})$

$T_1 \leftarrow \text{Parse}(\text{izquierda}(\text{exp}, \text{op}))$

$T_2 \leftarrow \text{Parse}(\text{derecha}(\text{exp}, \text{op}))$

retornar un árbol binario con op como rótulo de la raíz
y T_1 y T_2 como hijos izquierdo y derecho resp.



Parse(5+6-7)
 $\text{exp} = 5+6-7$
 $\text{op} = -$
 $\text{Izq}(\text{exp}, \text{op}) = 5+6$
 $\text{Der}(\text{exp}, \text{op}) = 7$
 $\text{Resultado} = t5 = \text{AB}(-, t3, t4)$

Parse(5+6)
 $\text{Exp} = 5+6$
 $\text{Op} = +$
 $\text{Izq}(\text{exp}, \text{op}) = 5$
 $\text{Der}(\text{exp}, \text{op}) = 6$
 $\text{Resultado} = t3 = \text{AB}(+, t1, t2)$

Parse(7)
 $\text{Exp} = 7$
 $\text{Resultado} = t4 = \text{AB}(7)$

Parse(5)
 $\text{Exp} = 5$
 $\text{Resultado} = t1 = \text{AB}(5)$

Parse(6)
 $\text{Exp} = 6$
 $\text{Resultado} = t2 = \text{AB}(6)$

Obtención del árbol binario de expresión aritmética totalmente parentizada

Ejemplo: $\text{exp} = (((4+5) * (6-1)) + (8*9))$

Algoritmo Parse(exp)

Si exp no contiene un operador **entonces** // por ej: $\text{exp}=4$

retornar un árbol binario hoja conteniendo rótulo de exp

Sino // por ej: $\text{exp} = (4+5)$

$\text{exp} \leftarrow$ Eliminar primer y último paréntesis de exp // $\text{exp} = 4+5$

$\text{op} \leftarrow$ operador_principal(exp) // $\text{op}=+$

$T_1 \leftarrow$ Parse(izquierda(exp, op)) // llama a Parse(4)

$T_2 \leftarrow$ Parse(derecha(exp, op)) // llama a Parse(5)

retornar un árbol binario con op como rótulo de la raíz // +

y T_1 y T_2 como hijos izquierdo y derecho resp. // 4 5

Nota: *operador_principal* recorre exp y usa un contador de paréntesis (de apertura incrementa y de cierre decrementa) para determinar cuándo encuentra el operador principal.

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ operador_principal(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz

y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Cont=0

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz

y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Encuentro (=> cont++ y avanzo

Cont=0

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Encuentro (=> cont++ y avanzo

Cont=1

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑ *Avanzo hasta) y cont--*

Cont=2

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑ *Avanzo hasta) y cont--*

Cont=1

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Avanzo hasta (y cont++

Cont=1

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Avanzo hasta) y cont--

Cont=2

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz

y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Avanzo, encuentro) y cont--

Cont=2

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Avanzo, encuentro) y cont--

Cont=1

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

↑

Avanzo, encuentro) y cont--

Cont=1

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)



Como cont=0 entonces el próximo carácter es el operador principal.

Cont=0

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

⇩ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)



El operador principal es + y hago op=+

Cont=0

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

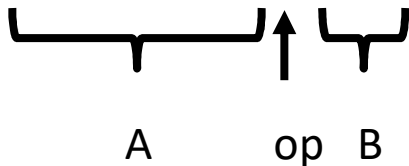
T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz
y T₁ y T₂ como hijos izquierdo y derecho resp.

exp = (((4+5) * (6-1)) + (8*9))

↓ Eliminar 1ero y ult ()

exp = ((4+5) * (6-1)) + (8*9)


A op B

Op=+

A= izquierda(exp,op) = ((4+5) * (6-1))

B = Derecha(exp,op) = (8*9)

Hay que llamar a la recursión con A y B.

Cont=0

Obtención del árbol binario de expresión aritmética totalmente parentizada

Algoritmo Parse(exp)

Si exp no contiene una operador **entonces**

retornar un árbol binario hoja conteniendo rótulo de exp

Sino

exp $\leftarrow$ Eliminar primer y último paréntesis de exp

op $\leftarrow$ último_operador(exp)

T₁ $\leftarrow$ Parse(izquierda(exp, op))

T₂ $\leftarrow$ Parse(derecha(exp, op))

retornar un árbol binario con op como rótulo de la raíz y T₁ y T₂ como hijos izquierdo y derecho resp.

Supongamos que:

T₁ = Parse(A)

T₂ = Parse(B)

Entonces Parse(exp) =

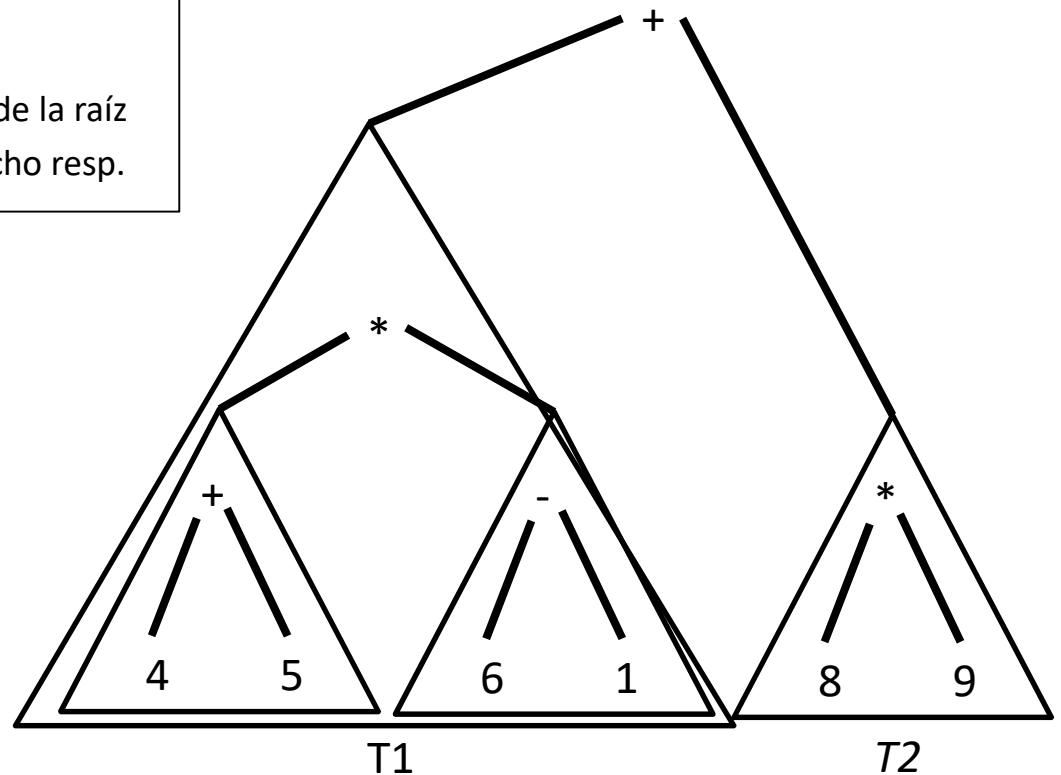
exp = (((4+5) * (6-1)) + (8*9))

↓ *Eliminar 1ero y ult ()*

exp = ((4+5) * (6-1)) + (8*9)

$\underbrace{((4+5) * (6-1))}_{A} \uparrow \underbrace{(8*9)}_B$
A op B

Cont=0



Evaluación de un árbol de expresión aritmética

Cada nodo en un árbol de expresión tiene un valor asociado:

- Si el nodo es externo, su valor es el de la variable o constante
- Si el nodo es interno, su valor está definido por la aplicación de la operación a los valores de sus hijos.

Algoritmo Evaluar(arbol_exp)

Si arbol_exp es una hoja **entonces**

retornar el valor del rótulo de arbol_exp

Sino

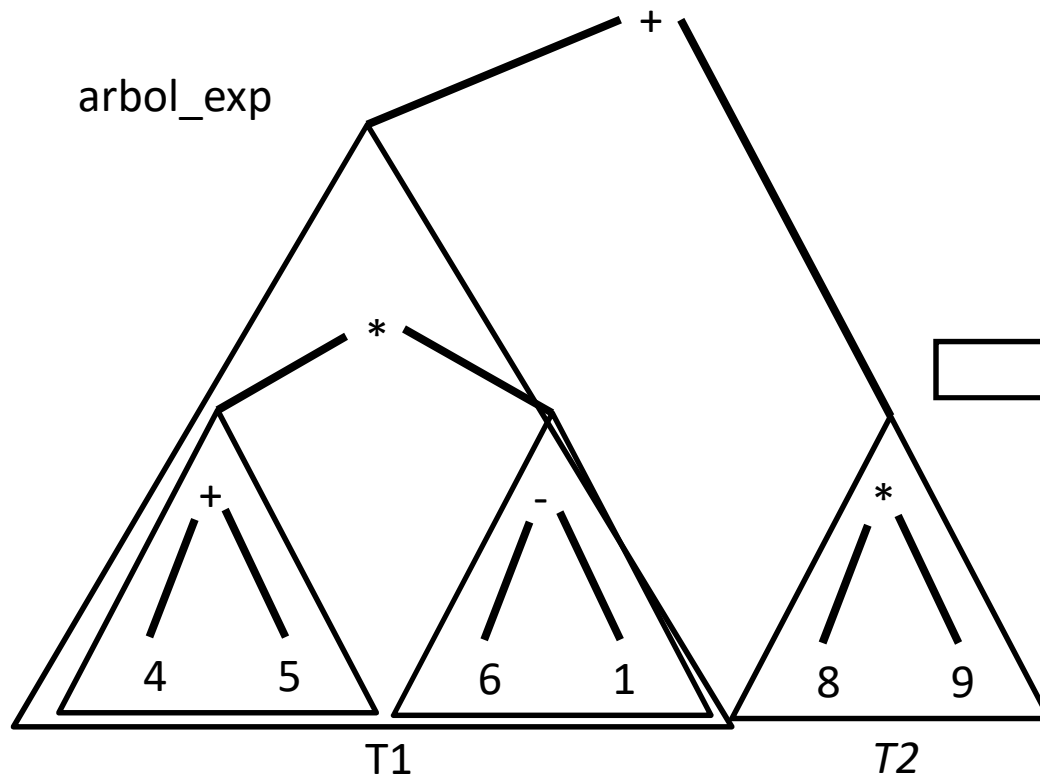
 op $\leftarrow$ rótulo de raíz de arbol_exp

$v_1 \leftarrow$ Evaluar(hijo izquierdo de arbol_exp)

$v_2 \leftarrow$ Evaluar(hijo derecho de arbol_exp)

retornar aplicar(op, v_1 , v_2) // Aplicar realiza la operación v_1 op v_2

En el ejemplo solo tenemos constantes. Si queremos tener variables, necesitamos un mapeo de string en número, para recuperar el valor de cada referencia a una variable.

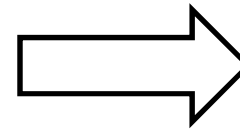


Eval(arbol_exp):

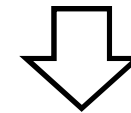
op= +

v1 = Eval(T1) = 45

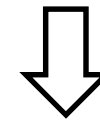
v2 = Eval(T2) = 72



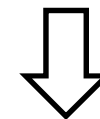
Aplicar(+, v1, v2)



Aplicar(+, 45, 72)

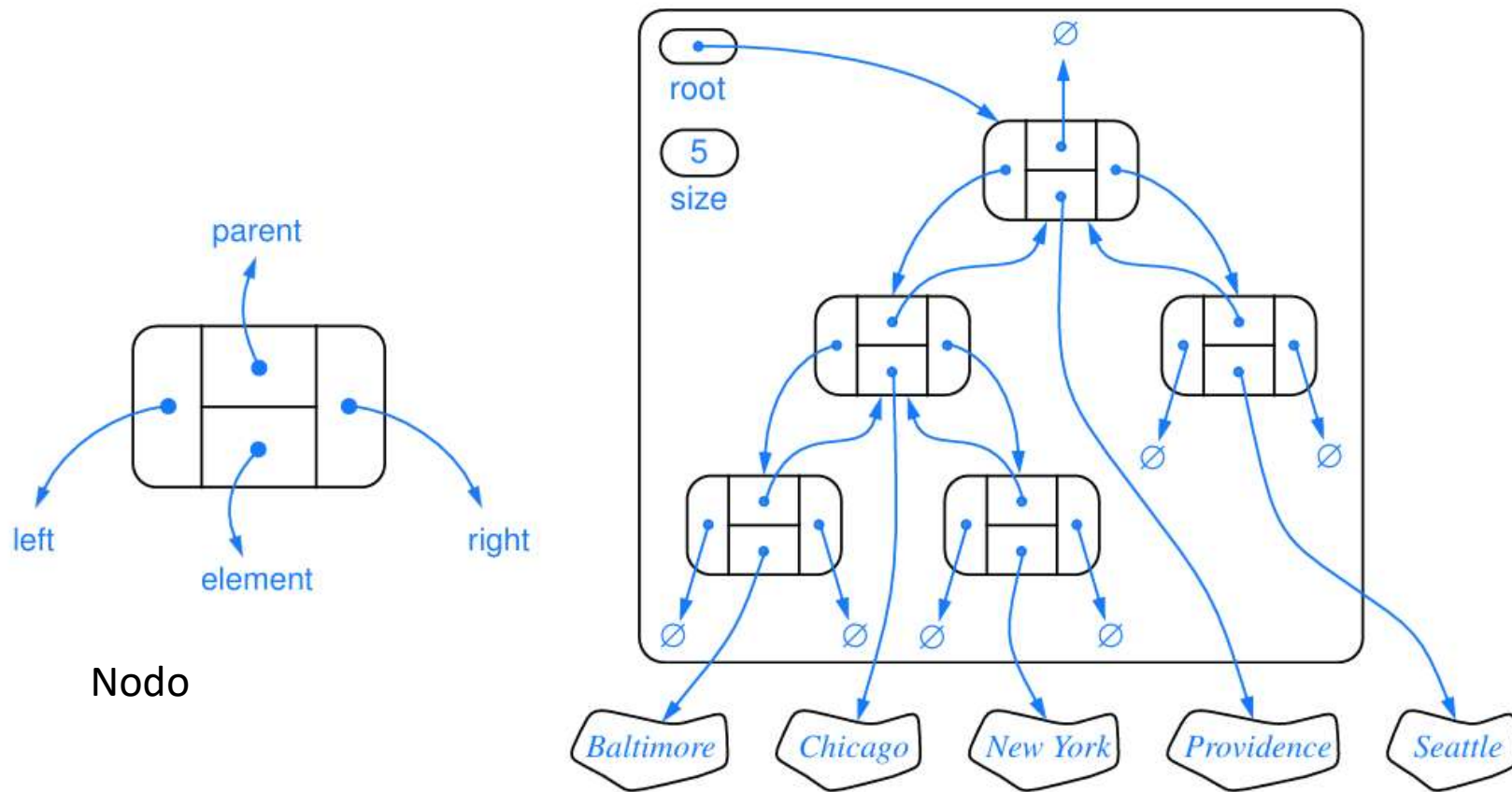


45 + 72



117

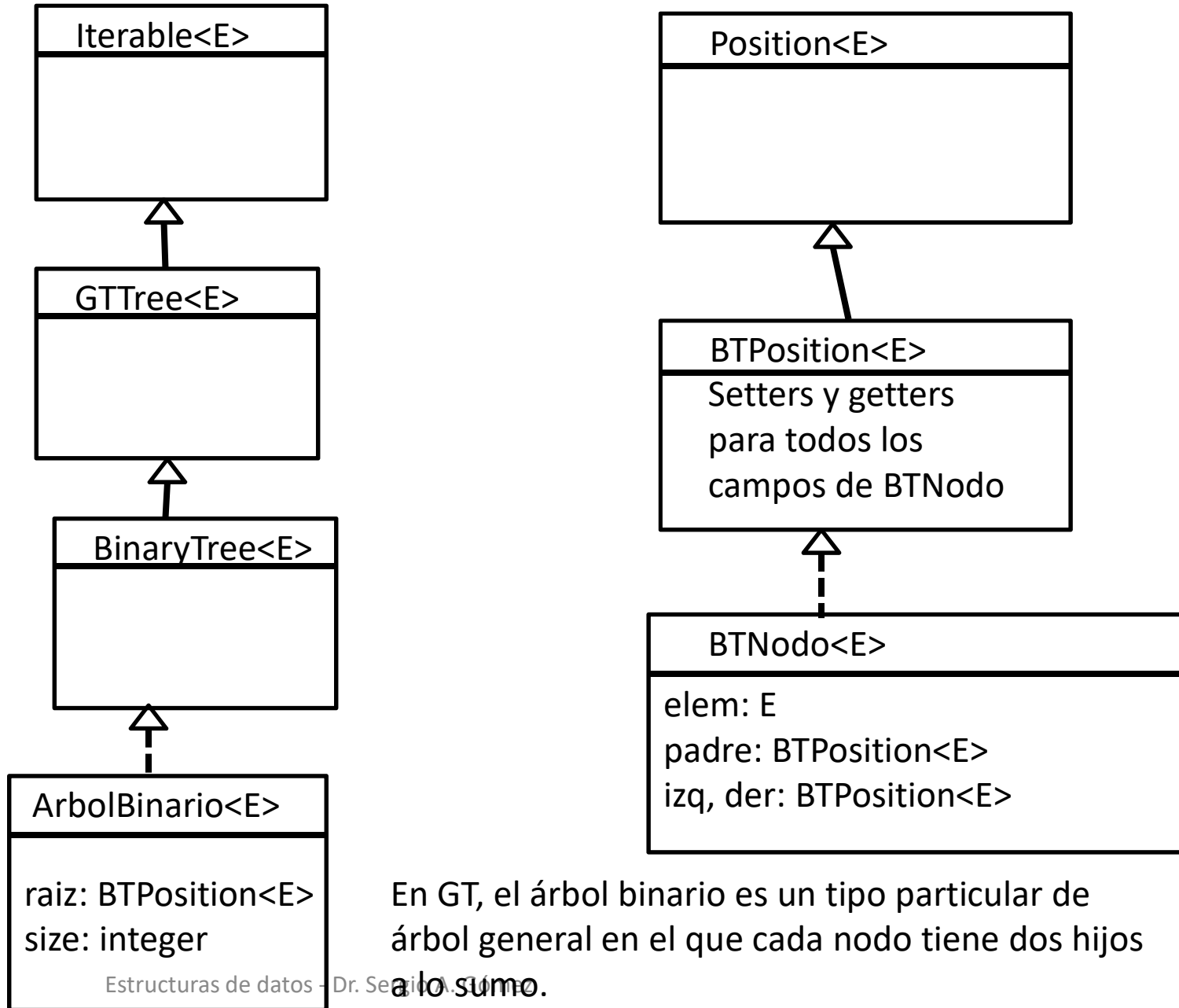
Estructura de datos del árbol binario



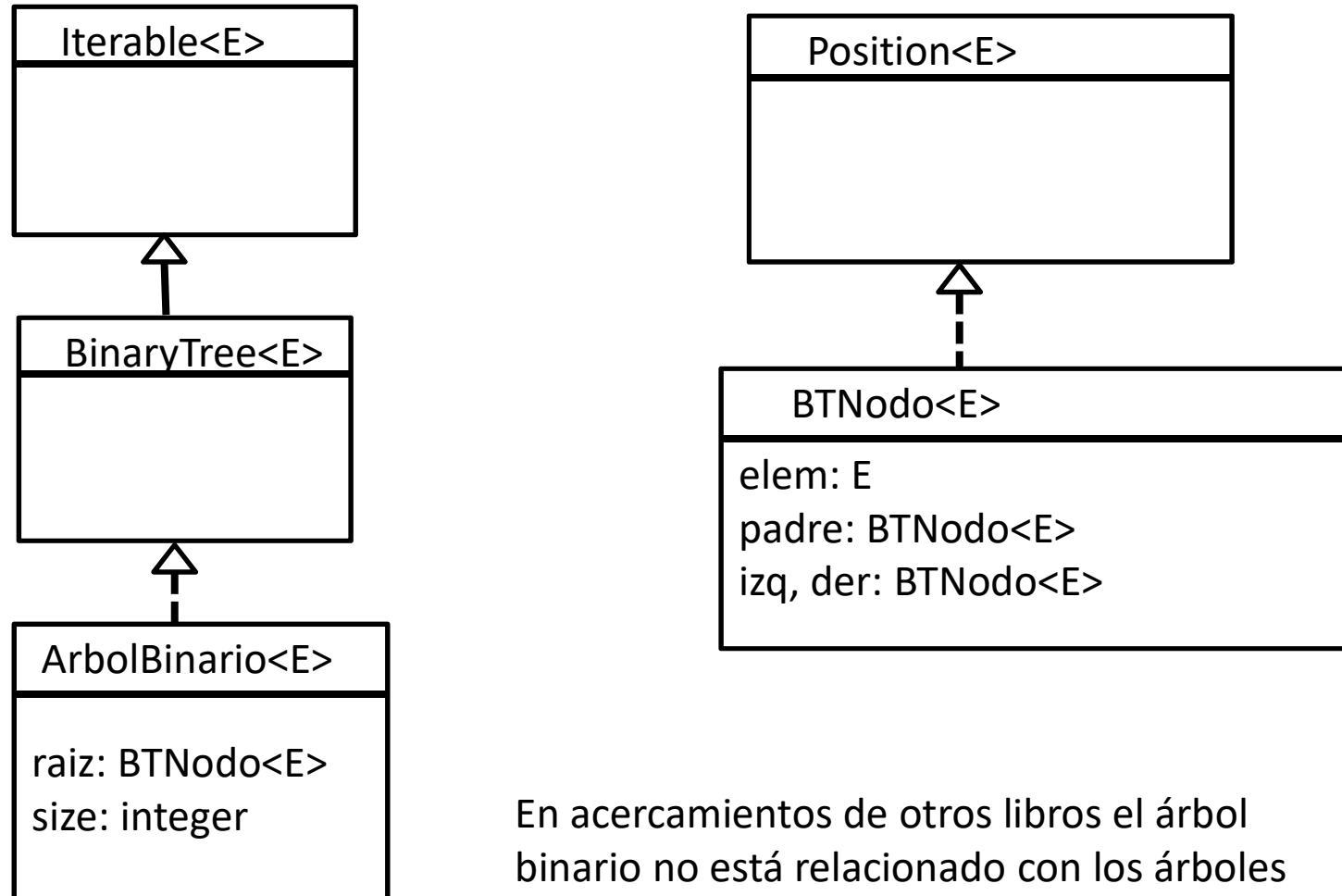
Tomado de GTW, pag. 323

Árbol binario

Árboles binarios en GT



Árboles binarios en otros libros



En acercamientos de otros libros el árbol binario no está relacionado con los árboles generales.

Esto facilita la implementación porque no hay que implementar las operaciones de Tree.

ADT Arbol Binario [GT]

El Árbol Binario (de acuerdo a GT) es una especialización (subinterfaz) de Tree que además soporta los métodos de acceso adicionales:

- `left(v)`: Retorna el hijo izquierdo de `v`, ocurre error si `v` no tiene hijo izquierdo
- `right(v)`: Retorna el hijo derecho de `v`, ocurre error si `v` no tiene hijo derecho
- `hasLeft(v)`: Testea si `v` tiene hijo izquierdo
- `hasRight(v)`: Testea si `v` tiene hijo derecho

Interfaz BinaryTree [GT]

```
public interface BinaryTree<E> extends Tree<E> {  
    // Lanzan InvalidPositionException (cuando v es inválida).  
    // Lanzan BoundaryViolationException (cuando ese hijo no existe):  
    public Position<E> left( Position<E> v );  
    public Position<E> right( Position<E> v );  
  
    // Lanzan InvalidPositionException (cuando v es inválida):  
    public boolean hasLeft( Position<E> v);  
    public boolean hasRight( Position<E> v);  
}
```


Estructura enlazada para el árbol binario

```
public interface BTPosition<E> extends Position<E> {  
    // agrega getters y setters para element, parent, left y right.  
}
```

```
public class BTNode<E> implements BTPosition<E> {  
    private E element;  
    private BTPosition<E> left, right, parent;
```

```
    public BTNode( E element, BTPosition<E> left, BTPosition<E> right,  
        BTPosition<E> parent ) {  
        this.element = element; this.left = left;  
        this.right = right; this.parent = parent;  
    }
```

```
    // setters y getters para element, left, right y parent.  
}
```

Implementación del árbol binario

```
public class ArbolBinarioEnlazado<E> implements BinaryTree<E> {
```

```
    protected BTPosition<E> raiz;
```

```
    protected int size;
```

En el libro se llama `LinkedBinaryTree`

```
    public ArbolBinarioEnlazado() { raiz = null; size = 0; }
```

```
    public int size() { return size; }  $T_{\text{size}}(n) = O(1)$ 
```

```
    public boolean hasLeft( Position<E> v) {
```

```
        BTPosition<E> n = checkPosition(v); // Propaga excepción de pos. inválida
```

```
        return n.getLeft() != null;  $T_{\text{hasLeft}}(n) = O(1)$ 
```

```
    }
```

```
    public Position<E> left(Position<E> v) {
```

```
        BTPosition<E> n = checkPosition(v); // Propaga excepción pos. inválida
```

```
        if( n.getLeft() == null ) throw new BoundaryViolationException( "..." );
```

```
        return n.getLeft();  $T_{\text{left}}(n) = O(1)$ 
```

```
    }
```

```
    public boolean isInternal(Position<E> v) {
```

```
        return hasLeft(v) || hasRight(v); // Propaga excepción de pos. inválida
```

```
    }  $T_{\text{isInternal}}(n) = O(1)$ 
```

Implementación del árbol binario (cont).

```
public Iterable<Position<E>> children( Position<E> v )
    throws InvalidPositionException {
    PositionList<Position<E>> hijos =
        new ListaDoblementeEnlazada<Position<E>>();
    if( hasLeft(v) ) hijos.addLast( left(v) );
    if( hasRight(v) ) hijos.addLast( right(v) );
    return hijos;
}
```

$$T_{\text{children}}(n) = O(2) = O(1)$$

Implementación del árbol binario (cont.)

- Métodos de modificación:
 - `addRoot(e)` (o `createRoot(e)`): Agrega un nodo raíz con rótulo `e`, error si ya hay raíz
 - `addLeft(v, e)`: Crea un nodo `w` hijo izquierdo de `v` con rótulo `e`, error si `v` ya tiene hijo izquierdo
 - `addRight(v,e)`: Crea un nodo `w` hijo derecho de `v` con rótulo `e`, error si `v` ya tiene hijo derecho
 - `remove(v)`: Elimina el nodo `v` (si `v` tiene un hijo, reemplaza a `v` por su hijo, si `v` tiene dos hijos entonces error).
 - `attach( v, T1, T2 )`: Setea `T1` como hijo izq de `v` y `T2` como hijo derecho de `v` (error si `v` no es hoja).
- Ver fragmento de código 7.16 a 7.20 en páginas 406 a 412 (implementación de `LinkedBinaryTree`).

Discusión: addRight

```
public Position<E> addRight(Position<E> v, E r) {  
    if (hasRight(v))  
        throw new InvalidOperationException("Ya tiene hijo  
derecho.");  
    BTNode<E> n = checkPosition(v);  
    BTNode<E> nuevo = new BTNode<E>(r, n);  
    n.setHijoDerecho(nuevo);  
    size++;  
    return nuevo;  
}
```

Discusión: attach en GT

```
/** Attaches two trees to be subtrees of an external node. */  
public void attach(Position<E> v, BinaryTree<E> T1, BinaryTree<E> T2)  
    throws InvalidPositionException {  
    BTPosition<E> vv = checkPosition(v);  
    if (isInternal(v))  
        throw new InvalidPositionException("Cannot attach from internal node");  
    if (!T1.isEmpty()) {  
        BTPosition<E> r1 = checkPosition(T1.root());  
        vv.setLeft(r1);  
        r1.setParent(vv);           // T1 should be invalidated  
    }  
    if (!T2.isEmpty()) {  
        BTPosition<E> r2 = checkPosition(T2.root());  
        vv.setRight(r2);  
        r2.setParent(vv);          // T2 should be invalidated  
    }  
}
```

Tomado de GT, pág. 412

Discusión: attach en GTG

```
/** Attaches trees t1 and t2 as left and right subtrees of external p. */
public void attach(Position<E> p, LinkedBinaryTree<E> t1,
                    LinkedBinaryTree<E> t2) throws IllegalArgumentException {
    Node<E> node = validate(p);
    if (isInternal(p)) throw new IllegalArgumentException("p must be a leaf");
    size += t1.size() + t2.size();
    if (!t1.isEmpty()) { // attach t1 as left subtree of node
        t1.root.setParent(node);
        node.setLeft(t1.root);
        t1.root = null;
        t1.size = 0;
    }
    if (!t2.isEmpty()) { // attach t2 as right subtree of node
        t2.root.setParent(node);
        node.setRight(t2.root);
        t2.root = null;
        t2.size = 0;
    }
}
```

Tomado de GTG, pág. 329

Tiempo de ejecución

Operación	Tiempo
size, isEmpty	$O(1)$
iterator, positions	$O(n)$
Replace	$O(1)$
Root, parent, children, left, right, sibling	$O(1)$
hasLeft, hasRight, isInternal, isExternal, isRoot	$O(1)$
addLeft, addRight, attach, remove	$O(1)$

Listados en árboles binarios

Algoritmo preorden(T, v)

Visitar v

si T.hasLeft(v) entonces

preorden(T, T.left(v))

si T.hasRight(v) entonces

preorden(T, T.right(v))

Algoritmo posorden(T, v)

si T.hasLeft(v) entonces

posorden(T, T.left(v))

si T.hasRight(v) entonces

posorden(T, T.right(v))

Visitar v

Algoritmo Inorden(T, v)

si T.hasLeft(v) entonces

Inorden(T, T.left(v))

Visitar v

si T.hasRight(v) entonces

Inorden(T, T.right(v))

Evaluación de expresión con recorrido posorden:

Algoritmo Eval(T, v) { Comienza en la raíz }

Si (T.isExternal(v)) retornar v.element()

sino

$R_1 \leftarrow \text{Eval}(T, T.\text{left}(v))$

$R_2 \leftarrow \text{Eval}(T, T.\text{right}(v))$

$op \leftarrow v.\text{element}()$

retornar aplicar(op, R_1 , R_2)

Nota: El listado inorder de un árbol de expresión aritmética reproduce la expresión original ¡Programar como ejercicio!

Representación del árbol binario con un arreglo

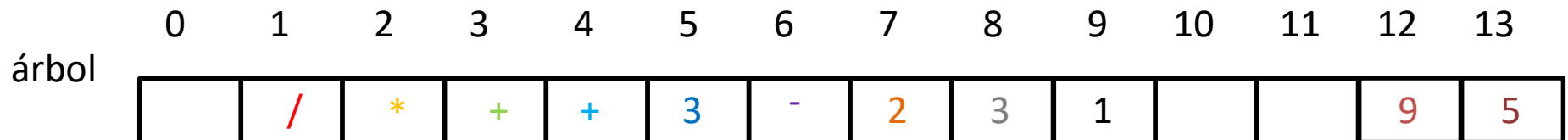
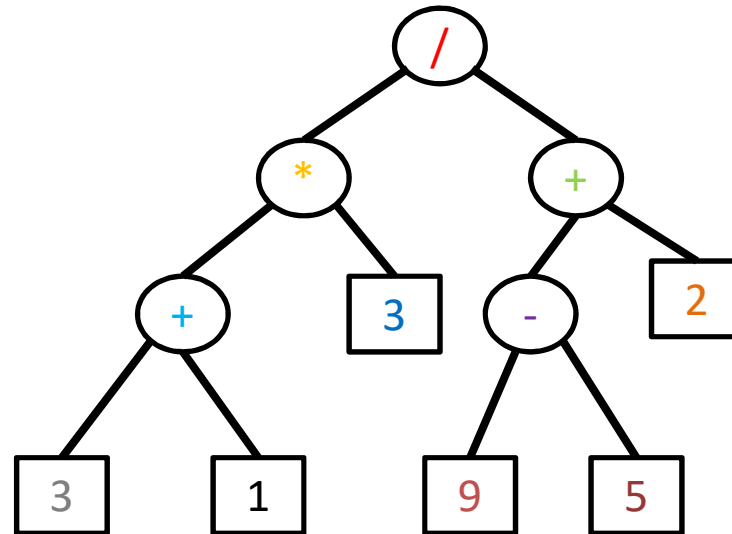
- La componente 0 no se usa
- La raíz se almacena en la componente 1
- El hijo izquierdo de la componente i se almacena en la componente $2*i$
- El hijo derecho de la componente i se almacena en la componente $2*i + 1$
- El padre de i está en la componente $i / 2$
- Nota: Puede haber componentes vacías si el árbol no está lleno (un árbol está lleno si tiene todos los nodos; un árbol binario lleno de altura h tiene $2^{h+1}-1$ nodos).

Representación con arreglos del árbol binario

Hijo_izquierdo(i) = 2i

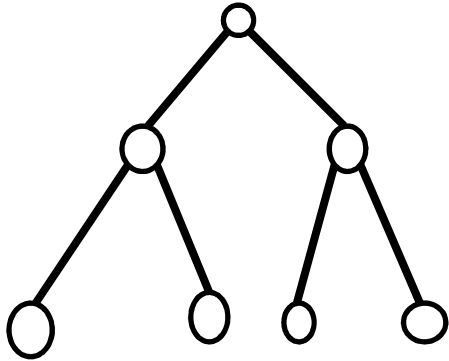
Hijo_derecho(i) = 2i+1

Padre(i) = i div 2

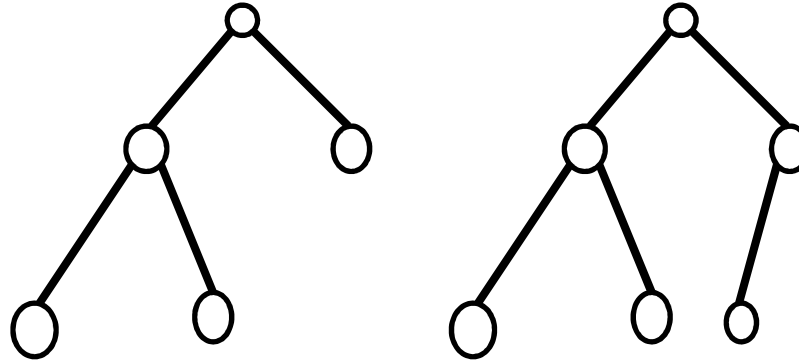


Ojo: La dimensión del vector crece exponencialmente en base 2 en la altura del árbol. Entonces esta implementación sólo es viable si el árbol está o bien lleno ó completo (el árbol está completo cuando está lleno hasta el penúltimo nivel y el último nivel se completa de izquierda a derecha, lo usaremos la clase que viene).

Árbol lleno versus árbol completo



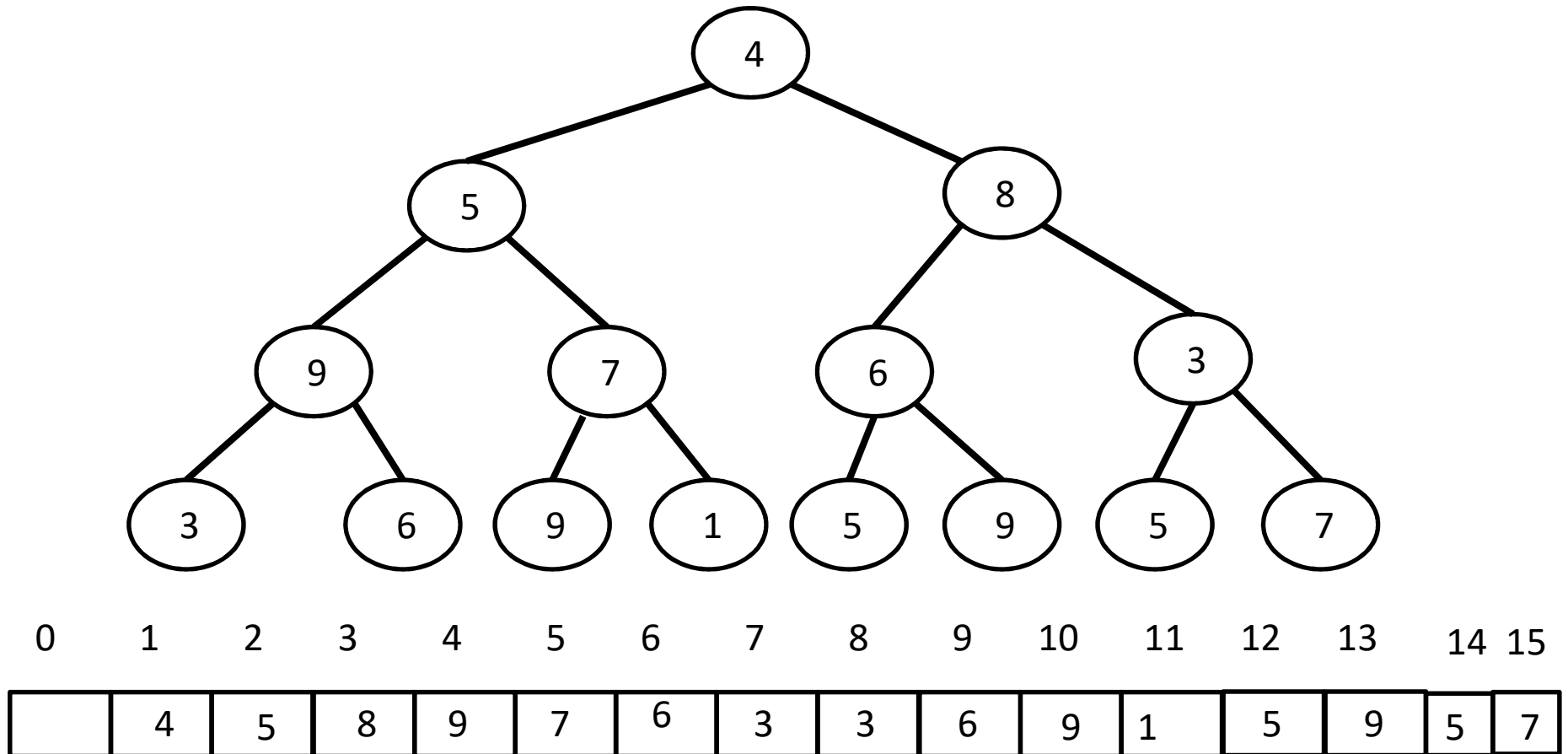
Árbol lleno de altura 2
tiene $2^{2+1}-1 = 7$ nodos



Dos posibles árboles
completos de altura 2

Propiedad: Un árbol binario lleno de altura h tiene $n = 2^{h+1}-1$ nodos.

Árbol lleno representado con arreglo



Notar que el arreglo se llena de acuerdo al orden inducido por el listado por niveles del árbol.

Bibliografía

- Capítulo 7 de Goodrich & Tamassia